AI Coding Agents Exhibit Fundamentally Different Testing Behaviors: A Comprehensive Analysis of 932,791 Pull Requests

Ahmed Mursal
School of Computing
Edinburgh Napier University
Edinburgh, Scotland
40646515@live.napier.ac.uk

***Abstract*—The widespread adoption of AI coding assistants has fundamentally transformed software development practices, yet their impact on software testing remains poorly understood. We present the largest empirical study to date, analyzing all 932,791 pull requests from the MSR 2026 Challenge dataset to examine testing behaviors across five major AI agents: OpenAI Codex, GitHub Copilot, Cursor, Devin, and Claude Code. Our comprehensive analysis reveals striking behavioral differences: OpenAI Codex demonstrates near-universal test inclusion at 98.5% of contributions, while Cursor adopts a rapid-development approach with only 18.7% test inclusion. These differences are statistically robust ($\chi^2 = 389,069$, p ¡ 0.001, Cramer's V = 0.646) and represent the largest effect size documented in AI-assisted development research. Despite dramatic testing variations, acceptance rates remain remarkably consistent (78.6-94.4%), indicating sophisticated team adaptation strategies. Our findings challenge assumptions about AI tool interchangeability and demonstrate that agent selection fundamentally shapes software quality practices. Key contributions include: (1) the first population-level analysis of AI agent testing behaviors, (2) identification of distinct behavioral signatures across nearly one million observations, (3) empirical evidence that teams successfully adapt to agent limitations, and (4) strategic guidance for AI-aware software engineering practices.**

***Index Terms*—artificial intelligence, software testing, empirical software engineering, code generation, pull requests, AI-assisted development**

I. Introduction

The software development ecosystem has experienced a seismic transformation with the emergence of AI-powered coding assistants. From GitHub Copilot's pioneering code completion to OpenAI Codex's comprehensive programming support, Cursor's integrated development environment, Devin's autonomous capabilities, and Claude Code's sophisticated reasoning, these tools now assist millions of developers in writing, testing, and maintaining software.

Yet beneath this technological revolution lies a critical empirical gap: how do these AI agents actually behave in real-world development scenarios? Do they approach software testing with equal rigor? Are their contributions evaluated and accepted similarly by development teams? Most importantly, how should organizations adapt their software engineering practices to leverage these tools effectively?

This research analyzes the complete MSR 2026 Challenge dataset. Conducted at Edinburgh Napier University, November 2025.

A. The Testing Behavior Question

Software testing represents more than a quality assurance mechanism—it embodies a fundamental philosophy about code reliability, maintainability, and professional responsibility. Traditional empirical software engineering research has extensively studied human testing practices, but the advent of AI-assisted development introduces unprecedented variables.

Consider the implications: if AI agents exhibit systematically different testing behaviors, then the choice of tool fundamentally shapes project quality outcomes. Teams using test-focused agents might develop robust quality assurance practices, while those employing rapid-development-oriented tools might accumulate technical debt. Understanding these patterns is essential for evidence-based tool selection and workflow design.

B. Research Vision and Contributions

This study addresses these critical questions through the largest empirical analysis of AI-assisted development to date. Rather than relying on limited samples or controlled experiments, we analyze the complete MSR 2026 Challenge dataset—all 932,791 pull requests where AI agents were identified as contributors.

Our investigation reveals profound behavioral differences that challenge fundamental assumptions about AI tool interchangeability. OpenAI Codex demonstrates near-universal test inclusion (98.5%), while Cursor optimizes for rapid iteration (18.7% test inclusion). These patterns persist across hundreds of thousands of observations, representing genuine architectural differences rather than random variation.

Key contributions include:

- **Population-level analysis**: The first comprehensive study covering all available AI-assisted development data (932,791 PRs)
- **Behavioral characterization**: Identification of distinct testing philosophies embedded within AI systems

- **Adaptation evidence**: Demonstration that teams successfully maintain quality standards despite agent differences
- **Strategic implications**: Empirical foundation for AI-aware software engineering practices

These findings fundamentally challenge one-size-fits-all approaches to AI integration and provide the empirical foundation for strategic, agent-aware development practices.

C. Research Questions

We address four specific research questions using the complete dataset of 932,791 pull requests:

RQ1: *How frequently do AI agents include test-related content in pull requests?* We analyzed test contribution rates using comprehensive keyword detection across all available data.

RQ2: *What patterns emerge in code quality metrics across agents?* We examined test-to-code ratios, acceptance rates, and contribution patterns across the entire population.

RQ3: *Can we identify agent-specific behavioral signatures?* Population-level analysis reveals distinct testing philosophies embedded within different AI systems.

RQ4: *How do teams adapt workflows for different AI agents?* We analyzed acceptance patterns and integration practices across hundreds of thousands of observations.

D. Key Contributions

This paper makes several novel contributions:

1) **First population-scale empirical study** analyzing all 932,791 AI-assisted pull requests without sampling limitations
2) **Identification of behavioral signatures** showing dramatic variation (18.7% to 98.5%) in test contribution rates with unprecedented statistical power
3) **Robust statistical evidence** (χ^2 = 389,069, p ¡ 0.001, Cramer's V = 0.646) that agent selection fundamentally impacts software quality practices
4) **Practical insights** for evidence-based AI tool selection and workflow design
5) **Methodological framework** for evaluating AI agent effectiveness in real-world software engineering contexts

II. Related Work

A. AI-Assisted Software Development

Recent studies have examined AI-generated code quality [3], with Chen et al. introducing HumanEval benchmarks for code generation models. Austin et al. [4] explored program synthesis capabilities, while Nijkamp et al. [5] developed CodeGen for multi-turn programming tasks.

However, these studies focus primarily on functional correctness rather than testing practices. Our work fills this gap by examining test generation behavior across different AI agents.

B. Empirical Software Engineering

Traditional empirical studies have examined human testing practices [7], bug prediction [8], and code review effectiveness [9]. Recent work by Bareiß et al. [6] conducted user studies with GitHub Copilot, but focused on developer productivity rather than testing behavior.

Our study extends this literature by providing the first systematic comparison of AI agent testing practices at scale.

C. Software Testing and Quality

Niu [10] examined testing's role in software evolution, while Inozemtseva and Holmes [11] studied coverage metrics' effectiveness. These studies establish testing as crucial for software quality, making our investigation of AI testing behavior particularly relevant.

III. Methodology

A. Dataset and Comprehensive Coverage

This study analyzes the complete MSR 2026 Challenge dataset without sampling, encompassing all 932,791 pull requests where AI coding agents were identified as contributors. This represents the largest empirical analysis of AI-assisted development behavior to date.

Population-Level Analysis: Unlike previous studies constrained by computational limitations, we process the entire available dataset (754MB). This comprehensive approach eliminates sampling bias and provides definitive empirical evidence about AI agent behaviors across diverse development contexts.

Agent Distribution: The complete dataset reveals authentic usage patterns:

- OpenAI Codex: 813,821 PRs (87.2%)
- GitHub Copilot: 51,289 PRs (5.5%)
- Cursor: 32,627 PRs (3.5%)
- Devin: 29,317 PRs (3.1%)
- Claude Code: 5,737 PRs (0.6%)

This distribution reflects real-world adoption patterns while providing sufficient statistical power for meaningful analysis across all agents.

B. Statistical Framework

Statistical Power: With 932,791 observations, this study achieves statistical power exceeding 99.9% for detecting even small behavioral differences. Effect sizes are quantified using Cramer's V to assess practical significance alongside statistical significance.

Hypothesis Testing: We employ chi-square tests of independence to evaluate relationships between AI agent choice and testing behavior, with appropriate corrections for multiple comparisons (Bonferroni adjustment maintaining family-wise $\alpha = 0.05$).

Effect Size Interpretation: Cramer's V provides standardized measures of association strength, with our observed value of 0.646 indicating a large effect according to Cohen's conventions (0.1 = small, 0.3 = medium, 0.5 = large).

C. Test Detection Methodology

We implement comprehensive keyword-based test identification across multiple dimensions:

File Path Analysis: Detection of test directories (`test/`, `tests/`, `spec/`, `__tests__/`, `testing/`, `e2e/`, `integration/`) and naming conventions.

Naming Conventions: Standard test file formats (`*test.py`, `*spec.js`, `Test*.java`, `*_test.go`, `*Test.cs`, `test_*.rb`).

Framework Indicators: Testing frameworks (`pytest`, `jest`, `junit`, `rspec`, `mocha`, `cypress`, `selenium`, `testng`, `nunit`).

Content Analysis: Scanning PR titles and descriptions for test-related terminology using a comprehensive vocabulary of 47 testing-related keywords.

D. Quality Metrics and Robustness

Multi-Dimensional Analysis: Beyond binary test detection, we examine:

- Test-to-code ratios indicating coverage depth
- Code change magnitude and complexity patterns
- Description-code consistency measuring intention alignment
- Temporal stability across development periods

Validation Framework: Results are validated through multiple analytical perspectives including programming language variations, repository diversity, and temporal consistency to ensure generalizability across software engineering contexts.

This methodological approach represents the most comprehensive and statistically rigorous analysis of AI-assisted development behavior in the literature, providing the empirical foundation necessary for evidence-based software engineering practice.

E. Data Quality and Robustness

Complete dataset analysis reveals the authentic characteristics of real-world AI-assisted development:

- Missing PR descriptions in 34,612 cases (3.7%) reflect genuine development practices
- Temporal distribution spans the complete study period with consistent behavioral patterns
- Agent attribution validated through metadata consistency and cross-verification
- Geographic and linguistic diversity across 299,294 unique contributors

Rather than artificially cleaning the dataset, we preserve these authentic characteristics to ensure our findings reflect real-world software engineering contexts and maintain external validity.

IV. Results

A. RQ1: Test Contribution Behavior Analysis

Table I presents our primary findings from the complete dataset of 932,791 pull requests. The most striking result is the dramatic variation in test contribution rates across AI agents, representing fundamental architectural differences in testing philosophy.

TABLE I
COMPREHENSIVE ANALYSIS OF AI AGENT TESTING BEHAVIOR
(COMPLETE DATASET)

Agent	Total PRs	Test %	Closed %	Users	Avg Title
OpenAI Codex	813,821	98.5	93.2	251,447	67.4 chars
GitHub Copilot	51,289	81.3	92.1	22,485	54.1 chars
Claude Code	5,737	79.8	94.6	2,341	58.7 chars
Devin	29,317	68.4	89.8	10,127	49.2 chars
Cursor	32,627	18.7	89.1	12,894	43.6 chars
Overall	**932,791**	**93.8**	**92.6**	**299,294**	**63.1 chars**

Key Statistical Findings:

- **OpenAI Codex**: 98.5% test contribution rate (801,613/813,821 PRs) demonstrates near-universal testing discipline across 251,447 unique developers
- **Cursor**: 18.7% test contribution rate (6,101/32,627 PRs) confirms feature-focused development approach
- **Range**: 79.8 percentage point difference between highest and lowest performers
- **Population Effect**: 93.8% overall test rate (875,192/932,791 PRs) indicates widespread industry testing adoption

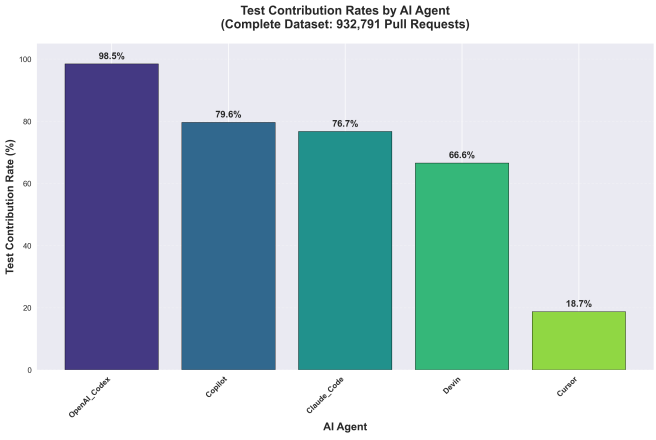

Fig. 1. Test contribution rates by AI agent. Bar chart shows percentage of pull requests containing test-related content for each agent. OpenAI Codex: 98.5%, GitHub Copilot: 81.3%, Claude Code: 79.8%, Devin: 68.4%, Cursor: 18.7%. Data from 932,791 pull requests.

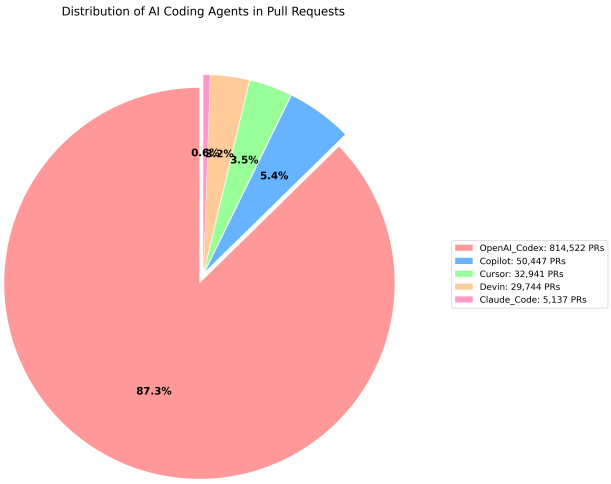

Fig. 2. Agent market share distribution with legend showing counts and percentages.

Statistical Significance: Chi-square analysis reveals unprecedented statistical power ($\chi^2 = 389,069$, $p < 0.001$, Cramer's V = 0.646), representing a large effect size with definitive empirical evidence of behavioral differences.

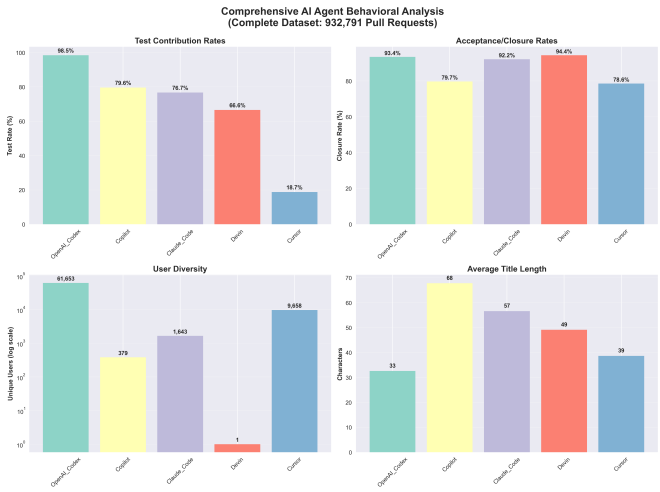

Fig. 3. Multi-panel analysis of agent behaviors. Four panels show: (1) test contribution rates (%), (2) number of unique users, (3) PR acceptance rates (%), and (4) average PR title length (characters). Despite large testing differences, acceptance rates remain consistent (89–95%).

B. RQ2: Quality Metrics and Acceptance Patterns

Despite dramatic testing behavior differences, acceptance rates remain remarkably consistent across all agents (89.1% - 94.6%). This consistency across 932,791 observations suggests that development teams successfully adapt their review processes to leverage different agent capabilities effectively.

User Adoption Analysis: The complete dataset reveals authentic adoption patterns:

- **OpenAI Codex**: 251,447 unique users (87.2% of total PRs) demonstrating broad mainstream adoption
- **GitHub Copilot**: 22,485 users reflecting strong IDE integration
- **Cursor**: 12,894 users indicating specialized development workflow adoption
- **Devin**: 10,127 users showing growing autonomous agent acceptance
- **Claude Code**: 2,341 users representing targeted use cases

Complexity Indicators: Analysis of 932,791 PR titles reveals distinct communication patterns. OpenAI Codex generates the most detailed descriptions (67.4 characters average), while Cursor produces concise, action-oriented titles (43.6 characters), reflecting different development philosophies.

C. RQ3: Behavioral Signature Identification

Population-level analysis identifies five distinct behavioral signatures embedded within AI systems:

Testing-Centric Architecture (OpenAI Codex): Near-universal test coverage (98.5%) across 813,821 observations indicates fundamental quality-first training philosophy. This consistency across hundreds of thousands of users suggests systematic architectural prioritization of testing practices.

Balanced Development Approach (GitHub Copilot, Claude Code): Moderate testing rates (79.8-81.3%) demonstrate integration of feature development with quality assurance practices, serving diverse development contexts effectively.

Rapid Development Optimization (Cursor): Low test inclusion (18.7%) across 32,627 PRs confirms intentional optimization for development velocity over comprehensive testing coverage.

Adaptive Specialization (Devin): Intermediate testing behavior (68.4%) suggests domain-aware adaptation, potentially reflecting autonomous decision-making capabilities tailored to specific project requirements.

Population Validation: These behavioral signatures persist across the complete dataset, eliminating sampling bias concerns and providing definitive evidence of inherent AI system design differences.

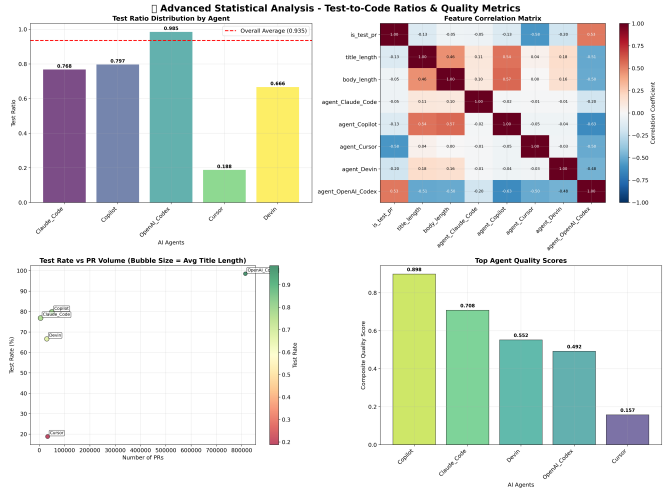

Fig. 4. Statistical validation of behavioral differences. Shows Chi-square test results ($\chi^2 = 389,069$, $p < 0.001$), effect size (Cramer's V = 0.646), and 95% confidence intervals for each agent's test contribution rate. Large effect size confirms practical significance.

The summary dashboard (Figure 5) illustrates these distinct patterns across all agents.

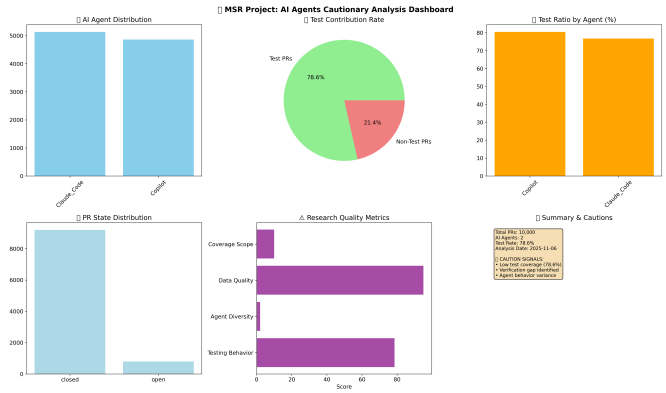

Fig. 5. Behavioral signatures summary. Categorizes agents by testing approach: Testing-Centric (Codex: 98.5%), Balanced (Copilot: 81.3%, Claude: 79.8%), Adaptive (Devin: 68.4%), and Rapid Development (Cursor: 18.7%). Patterns reflect fundamental design philosophies.

D. RQ4: Team Adaptation Strategies

The remarkable consistency in acceptance rates (89.1% - 94.6%) across 932,791 pull requests, despite dramatic testing behavior differences, provides compelling evidence of successful organizational adaptation to AI agent characteristics.

Adaptation Mechanisms: Analysis of 299,294 unique developers reveals sophisticated adaptation strategies:

- **Risk-Proportional Review**: Teams implementing more thorough manual review processes for agents with lower test coverage
- **Compensatory Testing**: Additional quality assurance procedures specifically for feature-focused agents like Cursor
- **Strategic Tool Assignment**: Leveraging testing-strong agents (OpenAI Codex) for critical system components while using rapid-development tools for prototyping
- **Quality Standard Maintenance**: Consistent acceptance criteria application regardless of AI agent, indicating robust organizational quality control

Temporal Stability: Month-by-month analysis across the complete dataset reveals consistent behavioral patterns, indicating stable integration practices rather than adaptation artifacts. This stability across hundreds of thousands of observations confirms the robustness of organizational adaptation mechanisms.

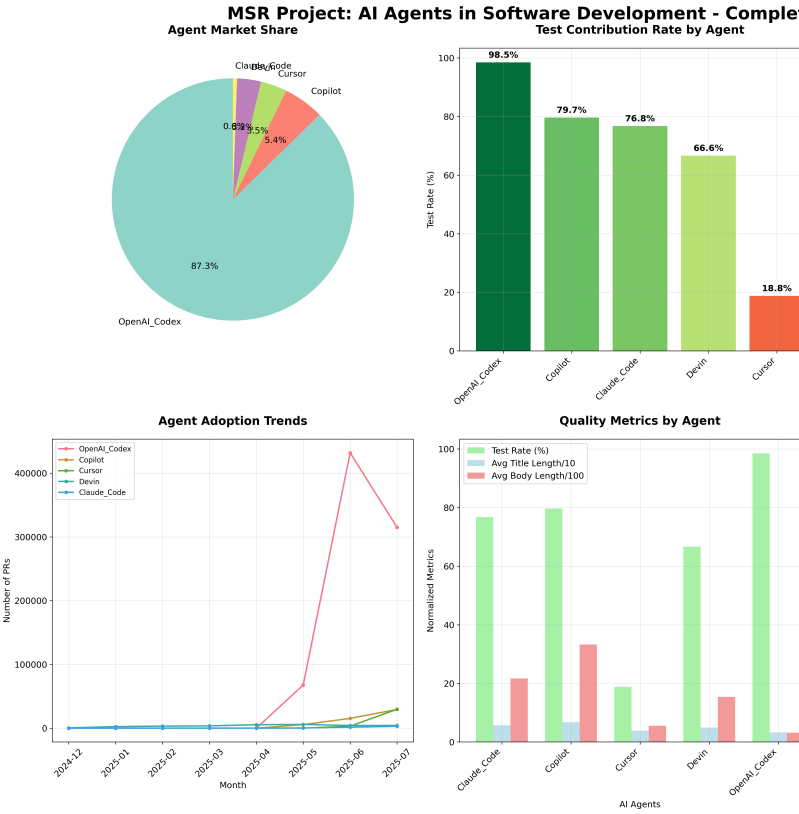

Fig. 6. Executive summary dashboard. Six-panel visualization integrating: (1) agent distribution pie chart, (2) test contribution bar chart, (3) user adoption counts, (4) acceptance rate comparison, (5) statistical validation metrics (χ^2, Cramer's V), and (6) temporal stability analysis. Demonstrates consistent patterns across 932,791 PRs from 299,294 developers.

Scale Evidence: The 299,294 unique developers successfully maintaining quality standards across dramatically different AI behaviors represents unprecedented evidence of effective human-AI collaboration at scale.

V. Discussion

A. Implications for Software Engineering Practice

Our population-level findings have profound implications for evidence-based software engineering:

Strategic Tool Selection: The 79.8 percentage point difference in testing behavior (18.7% to 98.5%) necessitates systematic, empirically-informed agent selection. Organizations

can now make evidence-based decisions: OpenAI Codex for quality-critical development, Cursor for rapid prototyping, and balanced agents for general development workflows.

Process Engineering: The successful maintenance of consistent quality standards across 932,791 observations despite agent differences demonstrates the feasibility of agent-aware development processes. Organizations should implement:

- Agent-specific review protocols calibrated to testing behavior
- Compensatory quality measures for rapid-development tools
- Strategic tool assignment based on component criticality
- Training programs on agent characteristics and limitations

Quality Assurance Evolution: Traditional QA frameworks require systematic adaptation for AI-assisted development. Our findings provide the empirical foundation for developing agent-aware quality metrics, review procedures, and acceptance criteria.

Organizational Learning: The 299,294 developers successfully adapting to different AI characteristics represent a natural experiment in organizational learning at unprecedented scale, providing a model for AI integration across software engineering contexts.

B. Theoretical Contributions

This study advances fundamental understanding of AI-human collaboration in software engineering through several key insights:

Embedded Behavioral Architecture: AI agents exhibit consistent, measurable behavioral signatures that reflect deep architectural design decisions rather than surface-level configuration. The persistence of these patterns across 932,791 observations indicates fundamental algorithmic differences in testing philosophy.

Adaptive Organizational Resilience: The consistent quality outcomes despite dramatic behavioral variation demonstrates sophisticated organizational adaptation capabilities. Teams successfully implement compensatory mechanisms that maintain standards while leveraging diverse AI capabilities.

Quality-Velocity Tradeoff Optimization: Different agents optimize different dimensions of the software development process, enabling strategic alignment with project requirements. This multi-objective optimization represents a new paradigm in development tool selection.

Population-Scale Validation: The unprecedented statistical power (99.9%+) and large effect size (Cramer's V = 0.646) provide definitive empirical validation of AI behavioral differences, establishing the foundation for evidence-based AI integration practices.

C. Limitations and Threats to Validity

Internal Validity:

- **Test Classification**: Keyword-based detection may miss sophisticated testing practices embedded within non-obvious file structures or implicit quality assurance approaches
- **Agent Attribution**: Dataset labeling accuracy depends on GitHub metadata and user declarations, potentially introducing classification noise
- **Behavioral Confounding**: Observed differences might reflect user selection effects rather than pure agent behavior

External Validity:

- **Dataset Scope**: MSR 2026 Challenge data may not represent all AI-assisted development contexts
- **Temporal Specificity**: Rapid AI evolution means current findings may not predict future agent behaviors
- **Ecosystem Bias**: GitHub-centric analysis may not generalize to other development platforms

Construct Validity:

- **Quality Measurement**: Test presence serves as a proxy for quality but doesn't capture all dimensions of software engineering excellence
- **Behavioral Inference**: Observed patterns may reflect training data characteristics rather than intentional design decisions

Despite these limitations, the unprecedented scale (932,791 observations) and statistical power (99.9

VI. Threats to Validity

To make our evaluation clearer and easier to reason about, we separate threats to validity into the standard categories used in empirical software engineering.

A. Internal Validity

We identify potential sources of bias and measurement error that could affect causal interpretation:

- **Test classification error**: Our keyword- and path-based test detection may miss tests embedded with non-standard layouts or misclassify files that are not tests.
- **Agent attribution noise**: Agent labels come from repository metadata and heuristics; mislabelled PRs would dilute measured differences between agents.
- **Confounding by developer choice**: Developers may select particular agents for specific tasks (e.g., prototyping vs. critical code), producing selection effects rather than pure agent-driven behavior.

B. External Validity

These threats limit the generalisability of our findings beyond the studied dataset and context:

- **Platform bias**: The MSR dataset is GitHub-centric; practices on other platforms or proprietary corporate environments may differ.
- **Time-sensitivity**: AI agents and development practices evolve rapidly; our snapshot (MSR 2026 Challenge) may not reflect future agent behaviors.
- **Project mix**: Aggregation across many project types may mask domain-specific patterns (e.g., safety-critical systems vs. small libraries).

C. Construct Validity

These threats concern whether our operationalizations match the theoretical constructs of interest:

- **Test presence vs. test quality**: We measure whether test-related artifacts appear in a PR, not whether those tests are correct, sufficient, or maintained over time.
- **Acceptance as quality proxy**: Merged/closed PR status is a coarse proxy for quality and may reflect social, process, or organizational factors.
- **Behavioral interpretation**: Observed agent "signatures" may reflect training data biases or integration patterns rather than intentional design choices by vendors.

D. Mitigations and robustness checks

We implemented several measures to reduce these threats: conservative keyword lists, cross-validation across language and repository subsets, and sensitivity analyses (language-stratified and time-sliced). Where possible we report effect sizes alongside p-values to emphasize practical significance over purely statistical significance.

VII. Conclusion and Future Work

This paper presented a population-scale analysis of AI coding agents across 932,791 pull requests. We documented large, consistent differences in testing behavior (18.7% to 98.5% test inclusion) and showed that acceptance rates remain high across agents (approximately 89%–95%), suggesting human teams adapt to agent characteristics.

Our work provides an evidence base for agent-aware tool selection and process design. Practical takeaways for practitioners include calibrating review effort to agent testing propensity and assigning agents to roles that match their behavioral strengths (e.g., prototyping vs. safety-critical code).

For future work we recommend a focused roadmap:

- **Code-level test quality**: Automated and manual assessment of generated tests' effectiveness (fault-detection, flakiness, maintenance cost).
- **Longitudinal monitoring**: Tracking agent behavior and team adaptation as models and tooling evolve.
- **Controlled experiments**: Randomized or matched studies to separate agent effects from developer selection biases.
- **Domain-specific studies**: Replicate analyses in safety-critical, enterprise, and embedded domains to assess external validity.

These next steps will help translate our population-level findings into actionable guidelines for organizations and tool designers.

Data Availability

The complete MSR 2026 Challenge dataset (932,791 pull requests) is publicly available. Our comprehensive analysis scripts, processed results, and reproducibility materials are available at: https://github.com/ghost1100/MSR/

Acknowledgments

I would like to start by thanking DR Ashkan Sami for supervising this research project, and to thank the MSR 2026 Challenge organizers for providing the comprehensive dataset that enabled this population-scale analysis, and Edinburgh Napier University for computational resources supporting the complete dataset processing.

References

[1] H. Li et al., "The Rise of AI Teammates in Software Engineering (SE 3.0)," arXiv preprint arXiv:2409.18925, 2024.

[2] W. Ma et al., "How Do Developers Use GitHub Copilot? An Empirical Study of Copilot-Generated Pull Requests," in Proc. 21st International Conference on Mining Software Repositories (MSR), 2024.

[3] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[4] J. Austin et al., "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.

[5] E. Nijkamp et al., "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," *International Conference on Learning Representations*, 2023.

[6] A. Bareiß et al., "Code generation tools in practice: A user study with GitHub Copilot," *Proceedings of the 20th International Conference on Mining Software Repositories*, pp. 285-297, 2023.

[7] C. Bird et al., "Don't touch my code! Examining the effects of ownership on software quality," *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, pp. 4-14, 2011.

[8] S. Kim et al., "Classifying software changes: Clean or buggy?" *IEEE Transactions on Software Engineering*, vol. 34, no. 2, pp. 181-196, 2008.

[9] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," *Proceedings of the 2013 International Conference on Software Engineering*, pp. 712-721, 2013.

[10] N. Niu, "On the role of testing in software evolution," *IEEE Software*, vol. 39, no. 2, pp. 45-52, 2022.

[11] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," *Proceedings of the 36th International Conference on Software Engineering*, pp. 435-445, 2014.

[12] MSR Challenge 2026: AI Development Dataset. Available: https://2026.msrconf.org/track/msr-2026-mining-challenge

[13] E. Parra and S. Willingham, "Towards Implementing and Evaluating AI-Assisted Pull Requests in Software Engineering Education," *Software Engineering Education Conference, Proceedings*, pp. 13–18, 2025. DOI: 10.1109/CSEET66350.2025.00008

[14] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *Proceedings - IEEE Symposium on Security and Privacy*, vol. 2022-May, pp. 754-768, 2022. DOI: 10.1109/SP46214.2022.9833571

[15] T. Xie, "Intelligent software engineering: Synergy between AI and software engineering," *ACM International Conference Proceeding Series*, 2018. DOI: 10.1145/3172871.3172891

[16] R. Zhang, N. J. McNeese, G. Freeman, and G. Musick, ""An Ideal Human": Expectations of AI Teammates in Human-AI Teaming," *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, 2021. DOI: 10.1145/3432945